

MISSÃO 02: O Arquiteto de Módulos (Classes e Pacotes)

Objetivo: Abandonar o "arquivo único" e construir um sistema com duas classes conversando entre si, organizadas em pacotes (pastas lógicas).

No mundo corporativo, um sistema de e-commerce não coloca as regras de Venda e as regras de Cliente no mesmo arquivo. Tudo é separado por "Bairros" (Pacotes). É hora de você organizar a sua cidade!

PASSO 1: Preparando a Fundação (Local)

Desta vez, vamos fazer o caminho inverso. Criaremos o projeto localmente primeiro, testaremos, e depois subiremos para a nuvem.

1. Abra o Git Bash.
2. Vá para a sua Área de Trabalho: ``cd ~/Desktop``
3. Crie uma pasta nova para o projeto de hoje e entre nela:
``mkdir meu-primeiro-sistema``
``cd meu-primeiro-sistema``
4. Ligue o "Motor do Git" para começar a vigiar essa pasta:
``git init``

PASSO 2: A Arquitetura de Pastas (O Domínio)

O Java exige que as pastas sigam o formato do pacote (domínio invertido).

1. Vamos criar a estrutura de pastas profunda com um único comando. No Git Bash, digite:
``mkdir -p src/br/com/meusistema/model``
``mkdir -p src/br/com/meusistema/main``
2. (🔥) O que você fez: Você criou uma pasta ``src`` (Source), que dentro tem a ``br``, que tem a ``com``, que tem a ``meusistema``, que se divide em dois "bairros": ``model`` (para os moldes) e ``main`` (para a execução).

PASSO 3: Criando os "Tijolos" (As Classes)

Vamos abrir o VS Code e escrever o código.

1. No terminal, abra o VS Code na raiz do projeto: ``code .``
2. No menu lateral do VS Code, navegue até a pasta ``src/br/com/meusistema/model``.
3. Clique com o Botão Direito na pasta ``model`` > New File (Novo Arquivo) > Chame de ``Carro.java``.
4. Digite o código da "Planta do Carro" e Salve (``Ctrl+S``):

```
package br.com.meusistema.model;

public class Carro {
    public String modelo;
    public String cor;

    public void buzinar() {
        System.out.println("O " + modelo + " " + cor + " fez:
BIIII BIIII!");
    }
}
```



PASSO 4: Criando o "Motorista" (A Classe Principal)

O `Carro` sozinho não faz nada. Precisamos da classe executável para "ligar" o carro.

1. Navegue até a pasta `src/br/com/meusistema/main`.
2. Crie um arquivo chamado `Sistema.java`.
3. Digite o código abaixo e Salve (`Ctrl+S`):

```
package br.com.meusistema.main;

// 🚨 OLHE O IMPORT AQUI! O Sistema precisa avisar que vai usar
o Carro do outro "bairro".
import br.com.meusistema.model.Carro;

public class Sistema {
    public static void main(String[] args) {
        System.out.println("Iniciando o sistema...");

        // Criando o objeto (Construindo o Carro 1)
        Carro meuCarro = new Carro();
        meuCarro.modelo = "Fusca";
        meuCarro.cor = "Azul";

        // Chamando o método (Comportamento)
        meuCarro.buzinar();

        // Desafio Rápido: Crie um "Carro 2", de outro modelo e
        cor, e faça ele buzinar também!
    }
}
```



PASSO 5: O Teste de Compilação

Como separamos em pacotes, o processo de compilar via terminal exige um truque.

1. Abra o Terminal do VS Code (ou o Git Bash).

2. Para compilar TODOS os arquivos de todas as pastas de uma vez, digite:
``javac src/br/com/meusistema/model/.java src/br/com/meusistema/main/.java``
3. Para rodar o programa (informando o pacote completo), digite:
``java -cp src br.com.meusistema.main.Sistema``

(✅ Vitória: Se a frase do Fusca azul buzinando apareceu, a sua arquitetura de pacotes Java está funcionando perfeitamente!)

PASSO 6: O Envio (Push para o GitHub)

Projeto rodando? Hora de guardar no currículo (Nuvem).

1. Vá ao GitHub e crie um novo repositório chamado ``meu-primeiro-sistema``. (⚠️ NÃO marque a opção Add a README file. Deixe ele nascer totalmente vazio!).
2. Volte ao terminal da sua máquina e prepare a foto:
``git add .``
``git commit -m "feat: cria estrutura de pacotes e classes Carro e Sistema"``
3. Conecte sua máquina à nuvem (copie o comando que o próprio GitHub sugere na tela preta dele, será algo como):
``git remote add origin https://github.com/SEU_USUARIO/meu-primeiro-sistema.git``
4. Mande para a nuvem:
``git push -u origin main``

Missão 02 Concluída! Você acabou de arquitetar um mini-sistema orientado a objetos com múltiplos pacotes, testou via terminal e subiu a versão final para a nuvem!